

FSM, экраны, callback-контракт и пользовательские флоу

Новый проект должен сохранить не только backend-ядро, но и готовую UX/FSM-архитектуру. Нельзя делать просто набор команд. Продукт должен работать как пошаговый Telegram Bot + Mini App с понятными экранами, состояниями, callback-кнопками и сохранением промежуточных параметров в FSM.

1. Обязательная FSM-структура

Создай отдельный файл:

```
bot/states.py
```

В нём должны быть группы состояний:

```
from aiogram.fsm.state import State, StatesGroup

class GenerationStates(StatesGroup):
    waiting_for_input = State()
    waiting_for_repeat_prompt = State()

    waiting_for_image = State()
    waiting_for_video = State()
    waiting_for_video_prompt = State()

    uploading_reference_images = State()
    uploading_reference_videos = State()
    confirming_reference_images = State()

    waiting_for_reference_video = State()
    waiting_for_video_start_image = State()

    waiting_for_motion_character_image = State()
    waiting_for_motion_video = State()

    waiting_for_avatar_audio = State()

    selecting_duration = State()
    selecting_aspect_ratio = State()
    selecting_quality = State()

    waiting_for_kling_negative_prompt = State()
    waiting_for_kling_cfg_scale = State()

    waiting_for_veo_seed = State()
    waiting_for_veo_watermark = State()
```

```
waiting_for_veo_extend_prompt = State()

waiting_for_omni_seed = State()
waiting_for_omni_audio_ids = State()
waiting_for_omni_character_ids = State()
waiting_for_omni_voice_base = State()
waiting_for_omni_voice_name = State()
waiting_for_omni_voice_description = State()
waiting_for_omni_example_dialogue = State()
waiting_for_omni_character_name = State()
waiting_for_omni_character_audio_ids = State()
```

```
class PaymentStates(StatesGroup):
    selecting_package = State()
    waiting_promo_code = State()
    confirming_payment = State()
    waiting_payment = State()
    waiting_partner_withdraw_requisites = State()
    waiting_partner_withdraw_amount = State()
    waiting_partner_exchange_amount = State()
```

```
class AdminStates(StatesGroup):
    waiting_broadcast_text = State()
    confirming_broadcast = State()
    waiting_user_id = State()
    waiting_partner_user_id = State()
    waiting_credits_amount = State()
    waiting_price_value = State()
    waiting_prompt_id = State()
    waiting_prompt_reject_reason = State()
    waiting_promo_code_value = State()
    waiting_ai_request = State()
    confirming_ai_action = State()
```

```
class BatchGenerationStates(StatesGroup):
    selecting_mode = State()
    selecting_preset = State()
    entering_prompts = State()
    uploading_references = State()
    confirming_batch = State()
    selecting_batch_count = State()
```

```
class ImageAnalyzerStates(StatesGroup):
    waiting_for_photo = State()
    waiting_for_video_prompt = State()
    waiting_for_photo_vk = State()
```

Правило: новые экраны не должны хранить данные в глобальных переменных. Все временные параметры пользователя хранятся в `FSMContext`.

2. Базовая структура FSM data

Для фото-флоу используй такую структуру:

```
def default_image_flow_data(reference_images=None,
img_flow_step="select_model") -> dict:
    return {
        "generation_type": "image",
        "img_service": "banana_pro",
        "img_ratio": "1:1",
        "img_count": 1,
        "img_quality": "2K",
        "img_nsfw_checker": False,
        "nsfw_enabled": False,
        "reference_images": list(reference_images or []),
        "img_flow_step": img_flow_step,
        "preset_id": "new",
        "user_prompt": "",
    }
```

Для видео-флоу используй такую структуру:

```
def default_video_flow_data(
    v_type="text",
    v_model="v3_pro",
    v_duration=5,
    v_ratio="16:9",
) -> dict:
    return {
        "generation_type": "video",
        "video_flow_step": "select_model",

        "v_type": v_type,
        "v_model": v_model,
        "v_duration": v_duration,
        "v_ratio": v_ratio,

        "v_image_url": None,
        "reference_images": [],
        "v_reference_videos": [],
        "avatar_audio_url": None,

        "user_prompt": "",
    }
```

```

    "grok_mode": "normal",
    "grok_resolution": "480p",

    "veo_generation_type": "TEXT_2_VIDEO",
    "veo_translation": True,
    "veo_resolution": "720p",
    "veo_seed": None,
    "veo_watermark": "",

    "kling_negative_prompt": "",
    "kling_cfg_scale": 0.5,

    "omni_resolution": "720p",
    "omni_seed": None,
    "omni_audio_ids": [],
    "omni_character_ids": [],
    "omni_base_voice": "achernar",
    "omni_voice_name": "",
    "omni_voice_description": "",
    "omni_example_dialogue": "",
    "omni_character_name": "",
    "omni_character_audio_ids": [],
}

```

3. Главный экран

Главное меню должно быть не списком команд, а основным хабом продукта.

Обязательные кнопки:

-  Открыть Mini App
-  Создать фото
-  Создать видео
-  Motion Control
-  Промпт по фото
-  Промпт по видео
-  Лента
-  Библиотека промптов
-  AI-помощник
-  Баланс
-  Поддержка
-  Партнёрам
- ... Ещё

Пример функции:

```

from aiogram.types import InlineKeyboardButton, WebAppInfo
from aiogram.utils.keyboard import InlineKeyboardBuilder

def get_main_menu_keyboard(user_credits: int = 0, mini_app_url: str | None =
None):
    builder = InlineKeyboardBuilder()

    if mini_app_url:
        builder.row(
            InlineKeyboardButton(
                text="🚀 Открыть Mini App",
                web_app=WebAppInfo(url=mini_app_url),
            )
        )

    builder.row(
        InlineKeyboardButton(text="📷 Создать фото",
callback_data="create_image_text_new"),
        InlineKeyboardButton(text="🎬 Создать видео",
callback_data="create_video_new"),
    )
    builder.row(
        InlineKeyboardButton(text="🎯 Motion Control",
callback_data="motion_control"),
        InlineKeyboardButton(text="📸 Промпт по фото",
callback_data="photo_to_prompt"),
    )
    builder.row(
        InlineKeyboardButton(text="🎞 Промпт по видео",
callback_data="video_to_prompt"),
        InlineKeyboardButton(text="📺 Лента", callback_data="menu_feed"),
    )
    builder.row(
        InlineKeyboardButton(text="📖 Библиотека промптов",
callback_data="menu_prompts"),
        InlineKeyboardButton(text="🤖 AI-помощник",
callback_data="menu_ai_assistant"),
    )
    builder.row(
        InlineKeyboardButton(text=f"💰 Баланс: {user_credits}",
callback_data="menu_balance"),
        InlineKeyboardButton(text="💬 Поддержка",
callback_data="menu_support"),
    )
    builder.row(
        InlineKeyboardButton(text="👉 Партнёрам",
callback_data="menu_partner"),
        InlineKeyboardButton(text="... Ещё", callback_data="ux_more"),
    )

```

```
return builder.as_markup()
```

4. Фото-флоу

Основной сценарий:

Главное меню

-  Создать фото
- выбор модели
- загрузка/пропуск референсов
- экран настроек
- ввод prompt
- проверка баланса
- списание кредитов
- создание generation_task
- вызов provider service
- ожидание webhook/polling
- выдача результата
- кнопки результата: повторить / опубликовать / в библиотеку / анимировать

Callback-контракт:

```
create_image_text_new
image_change_model
model_banana_pro
model_banana_2
model_nano_banana_2_lite
model_seedream_edit
model_grok_i2i
model_flux_pro
model_wan_27

img_ref_continue_new
ref_skip_new
ref_saved_library

img_ratio_1_1
img_ratio_16_9
img_ratio_9_16
img_ratio_4_3
img_ratio_3_4

img_quality_2k
img_quality_4k
img_quality_basic
```

```
img_quality_high
```

```
img_count_1
```

```
img_count_2
```

```
img_count_4
```

```
img_count_6
```

Пример входа в фото-флоу:

```
@router.callback_query(F.data == "create_image_text_new")
async def show_create_image_text_menu(callback: types.CallbackQuery, state:
FSMContext):
    await
    state.update_data(**default_image_flow_data(img_flow_step="select_model"))
    await show_image_model_selection_screen(callback, state)
    await callback.answer()
```

Пример экрана выбора модели:

```
async def show_image_model_selection_screen(callback: types.CallbackQuery,
state: FSMContext):
    data = await state.get_data()
    current_service = data.get("img_service", "banana_pro")

    text = (
        "📷 <b>Создание фото</b>\n\n"
        "<b>Шаг 1. Выберите модель</b>\n"
        "После выбора модели бот покажет следующий шаг."
    )

    await callback.message.edit_text(
        text,
        reply_markup=get_image_model_selection_keyboard(current_service),
        parse_mode="HTML",
    )

    await state.set_state(GenerationStates.waiting_for_input)
```

Пример выбора модели:

```
@router.callback_query(F.data == "model_banana_pro")
async def select_banana_pro(callback: types.CallbackQuery, state:
FSMContext):
    await state.update_data(
        img_service="banana_pro",
        img_quality="2K",
        img_flow_step="upload_refs",
    )
```

```

await show_image_reference_upload_screen(callback, state)
await callback.answer("Nano Banana Pro выбран")

```

Пример экрана настроек фото:

```

async def show_image_creation_screen(message_or_callback, state: FSMContext,
edit: bool = True):
    data = await state.get_data()

    text = (
        "📷 <b>Создание фото</b>\n\n"
        f"🤖 Модель: <code>{data.get('img_service')}</code>\n"
        f"📐 Формат: <code>{data.get('img_ratio')}</code>\n"
        f"🔗 Референсы: <code>{len(data.get('reference_images', []))}</"
code>\n"
        f"📄 Количество: <code>{data.get('img_count', 1)}</code>\n\n"
        "Напишите prompt одним сообщением."
    )

    keyboard = get_create_image_keyboard(
        current_service=data.get("img_service", "banana_pro"),
        current_ratio=data.get("img_ratio", "1:1"),
        current_count=data.get("img_count", 1),
        num_refs=len(data.get("reference_images", [])),
        img_quality=data.get("img_quality", "2K"),
    )

    target = message_or_callback.message if hasattr(message_or_callback,
"message") else message_or_callback

    if edit and hasattr(target, "edit_text"):
        await target.edit_text(text, reply_markup=keyboard,
parse_mode="HTML")
    else:
        await target.answer(text, reply_markup=keyboard, parse_mode="HTML")

    await state.set_state(GenerationStates.waiting_for_input)

```

Пример обработки prompt:

```

@router.message(GenerationStates.waiting_for_input, F.text)
async def handle_generation_prompt(message: types.Message, state:
FSMContext):
    data = await state.get_data()

    if data.get("generation_type") == "image":
        await run_image_generation(message, state,
prompt=message.text.strip())
    return

```



```

if data.get("generation_type") == "video":
    await run_video_generation(message, state,
prompt=message.text.strip())
    return

await message.answer("Выберите создание фото или видео в главном меню.")

```

5. Видео-флоу

Основной сценарий:

Главное меню

- 🎬 Создать видео
- выбор модели
- выбор типа и медиа
- настройки видео
- ввод prompt
- проверка баланса
- списание кредитов
- generation_task
- provider service
- webhook/polling
- выдача результата

Типы видео:

text	= Текст → Видео
imgtxt	= Фото + Текст → Видео
video	= Видео + Текст → Видео
avatar	= Аватар + Аудио → Видео
motion	= Motion Control
audio	= Gemini Omni Audio ID
character	= Gemini Omni Character ID

Callback-контракт:

```

create_video_new
video_change_model
video_change_media
video_media_continue
video_media_skip

v_model_v3_pro
v_model_v3_std

```

```
v_model_v26_pro
v_model_grok_imagine
v_model_grok_imagine_v15
v_model_seedance_2
v_model_gemini_omni
v_model_veo3
v_model_veo3_fast
v_model_veo3_lite
v_model_glow

v_type_text
v_type_imgtxt
v_type_video
v_type_avatar
v_type_motion

ratio_16_9
ratio_9_16
ratio_1_1
video_dur_4
video_dur_5
video_dur_6
video_dur_8
video_dur_10
video_dur_15

kling_negative_prompt_edit
kling_cfg_scale_edit

veo_translation_toggle
veo_resolution_720p
veo_resolution_1080p
veo_resolution_4k
veo_seed_edit
veo_watermark_edit

omni_mode_video
omni_mode_audio
omni_mode_character
omni_resolution_720p
omni_resolution_1080p
omni_resolution_4k
omni_seed_edit
omni_audio_ids_edit
omni_character_ids_edit
```

Пример входа в видео-флой:

```
@router.callback_query(F.data == "create_video_new")
async def show_create_video_menu(callback: types.CallbackQuery, state:
```

```

FSMContext):
    await state.update_data(**default_video_flow_data())
    await state.update_data(video_flow_step="select_model")
    await show_video_model_selection_screen(callback, state)
    await callback.answer()

```

Пример экрана выбора видео-модели:

```

async def show_video_model_selection_screen(callback: types.CallbackQuery,
state: FSMContext):
    data = await state.get_data()
    current_model = data.get("v_model", "v3_pro")

    text = (
        "🎬 <b>Создание видео</b>\n\n"
        "<b>Шаг 1. Выберите модель</b>\n"
        "После выбора модели бот покажет доступные типы, медиа и настройки."
    )

    await callback.message.edit_text(
        text,
        reply_markup=get_video_model_selection_keyboard(current_model),
        parse_mode="HTML",
    )

    await state.set_state(GenerationStates.waiting_for_input)

```

Пример выбора модели:

```

@router.callback_query(F.data.startswith("v_model_"))
async def select_video_model(callback: types.CallbackQuery, state:
FSMContext):
    model = callback.data.replace("v_model_", "", 1)

    await state.update_data(
        v_model=model,
        video_flow_step="media",
    )

    if model == "gemini_omni":
        await show_gemini_omni_mode_screen(callback, state)
    else:
        await show_video_media_screen(callback, state)

    await callback.answer()

```

Пример экрана медиа:

```

async def show_video_media_screen(callback: types.CallbackQuery, state:
FSMContext):
    data = await state.get_data()

    current_model = data.get("v_model", "v3_pro")
    current_v_type = data.get("v_type", "text")

    text = (
        "🎬 <b>Создание видео</b>\n\n"
        "<b>Шаг 2. Тип и медиа</b>\n"
        f"Модель: <code>{current_model}</code>\n"
        f"Тип: <code>{current_v_type}</code>\n\n"
        "Выберите тип генерации и загрузите нужные файлы, если они"
        "требуются."
    )

    await callback.message.edit_text(
        text,
        reply_markup=get_video_media_step_keyboard(
            current_v_type=current_v_type,
            current_model=current_model,
            has_start_image=bool(data.get("v_image_url")),
            reference_image_count=len(data.get("reference_images", [])),
            reference_video_count=len(data.get("v_reference_videos", [])),
            has_avatar_audio=bool(data.get("avatar_audio_url")),
        ),
        parse_mode="HTML",
    )

    await state.set_state(GenerationStates.waiting_for_video_prompt)

```

Пример экрана настроек видео:

```

async def show_video_creation_screen(callback: types.CallbackQuery, state:
FSMContext):
    data = await state.get_data()

    text = (
        "🎬 <b>Создание видео</b>\n\n"
        f"✏️ Тип: <code>{data.get('v_type')}</code>\n"
        f"📺 Модель: <code>{data.get('v_model')}</code>\n"
        f"⌚ Длительность: <code>{data.get('v_duration')}</code> сек</code>\n"
        f"📏 Формат: <code>{data.get('v_ratio')}</code>\n\n"
        "Напишите prompt одним сообщением."
    )

    await callback.message.edit_text(
        text,
        reply_markup=get_create_video_keyboard(

```

```

        current_v_type=data.get("v_type", "text"),
        current_model=data.get("v_model", "v3_pro"),
        current_ratio=data.get("v_ratio", "16:9"),
        current_duration=data.get("v_duration", 5),
        current_kling_negative_prompt=data.get("kling_negative_prompt",
        ""),
        current_kling_cfg_scale=float(data.get("kling_cfg_scale", 0.5)),
        current_veo_resolution=data.get("veo_resolution", "720p"),
        current_omni_resolution=data.get("omni_resolution", "720p"),
    ),
    parse_mode="HTML",
)

await state.set_state(GenerationStates.waiting_for_video_prompt)

```

6. Референсы

Референсы — отдельная обязательная часть продукта.

Нужно поддержать:

Фото-референсы:

- загрузка photo
- загрузка document image/jpeg, image/png, image/webp
- минимальная сторона изображения
- сохранение в static/uploads или S3-совместимое хранилище
- выдача публичного URL
- лимит по модели
- библиотека сохранённых референсов пользователя

Видео-референсы:

- загрузка video
- лимит по модели
- проверка размера/длительности
- сохранение публичного URL

Обязательные callback'и:

```

ref_skip_new
img_ref_continue_new
vid_ref_continue_new
ref_saved_library
savedref_nav_{index}
savedref_use_{reference_id}

```

```
saveref_delete_{reference_id}_{index}  
saveref_close
```

Пример:

```
@router.message(GenerationStates.uploading_reference_images)  
async def handle_reference_image_upload(message: types.Message, state:  
FSMContext):  
    data = await state.get_data()  
    refs = list(data.get("reference_images", []))  
  
    public_url = await save_reference_from_telegram_message(message)  
  
    if not public_url:  
        await message.answer("Не удалось сохранить фото. Отправьте JPG, PNG  
или WEBP.")  
        return  
  
    refs.append(public_url)  
    await state.update_data(reference_images=refs)  
  
    await message.answer(  
        f"✅ Референс добавлен: {len(refs)}",  
        reply_markup=get_reference_images_upload_keyboard(  
            current_count=len(refs),  
            max_count=9,  
            preset_id="new",  
        ),  
    )
```

7. Оплаты и баланс

Платёжный флоу тоже должен быть FSM-driven.

Сценарий:

```
Главное меню  
→ Баланс  
→ Пополнить  
→ выбор пакета  
→ ввод промокода, если нужно  
→ выбор способа оплаты  
→ создание pending transaction  
→ выдача payment URL  
→ webhook провайдера  
→ проверка подписи
```

- начисление кредитов
- уведомление пользователя

Callback-контракт:

```
menu_balance
menu_topup
choose_pay_{package_id}
topup_enter_promo
topup_remove_promo
buy_stars_{package_id}
buy_crypto_{package_id}
buy_yookassa_{package_id}
buy_lava_{package_id}
check_payment_{transaction_id}
```

Пример:

```
@router.callback_query(F.data == "menu_topup")
async def show_topup(callback: types.CallbackQuery, state: FSMContext):
    await state.set_state(PaymentStates.selecting_package)

    await callback.message.edit_text(
        "👉 <b>Пополнение баланса</b>\n\nВыберите пакет:",
        reply_markup=get_payment_packages_keyboard(load_payment_packages()),
        parse_mode="HTML",
    )

    await callback.answer()
```

8. Админские экраны

Админка должна быть отдельным экранным флоу, а не набором скрытых команд.

Обязательные разделы:

-  Статистика
-  Пользователи
-  Партнёры
-  Финансы/рефы
-  Цены
-  Промокоды
-  Промпты
-  ИИ-админ
-  Рассылка

-  Подписка на канал
-  Главное меню

Callback-контракт:

```
admin_stats
admin_users
admin_partners
admin_finance
admin_prices
admin_promocodes
admin_prompts
admin_ai
admin_broadcast
admin_required_subscription_toggle
admin_back
```

Пример:

```
@router.callback_query(F.data == "admin_broadcast")
async def admin_broadcast_start(callback: types.CallbackQuery, state:
FSMContext):
    await state.set_state(AdminStates.waiting_broadcast_text)
    await callback.message.answer(
        "⚙️ Отправьте текст рассылки одним сообщением."
    )
    await callback.answer()
```

9. Feed, repeat, remix

После результата генерации должны быть кнопки пост-действий:

-  Повторить
-  Новый prompt
-  Анимировать
-  В ленту
-  В библиотеку
-  /  Показать prompt
-  /  Показать референсы

Сценарий повтора:

Результат генерации
→ Повторить

- восстановить request_data из generation_tasks
- положить параметры в FSM
- показать экран повтора
- разрешить заменить prompt
- разрешить добавить/очистить референсы
- запустить новую generation_task

Пример:

```
@router.callback_query(F.data.startswith("repeat_image_"))
async def repeat_image_generation(callback: types.CallbackQuery, state:
FSMContext):
    task_id = callback.data.replace("repeat_image_", "", 1)
    task = await get_task_by_id(task_id)

    if not task:
        await callback.answer("Не удалось найти генерацию.", show_alert=True)
        return

    request_data = json.loads(task.request_data or "{}")

    await state.clear()
    await state.update_data(
        generation_type="image",
        img_service=request_data.get("img_service", task.model or
"banana_pro"),
        img_ratio=request_data.get("img_ratio", task.aspect_ratio or "1:1"),
        img_count=1,
        img_quality=request_data.get("img_quality", "2K"),
        reference_images=request_data.get("source_reference_images", []),
        repeat_source_task_id=task_id,
        repeat_prompt=request_data.get("prompt", task.prompt or ""),
        img_flow_step="configure",
    )

    await state.set_state(GenerationStates.waiting_for_input)
    await show_image_creation_screen(callback, state)
    await callback.answer("Можно изменить prompt или запустить повтор.")
```

10. Router order

Порядок подключения router'ов важен. Специфичные обработчики должны идти раньше общих.

Пример:

```
def setup_dispatcher(storage):
    dp = Dispatcher(storage=storage)

    dp.include_router(generation_router)
    dp.include_router(image_analyzer_router)
    dp.include_router(admin_router)
    dp.include_router(payments_router)
    dp.include_router(batch_generation_router)
    dp.include_router(common_router)

    return dp
```

Правило: `common_router` подключать последним, чтобы он не перехватывал callback'и и текст, предназначенные для FSM-флоу.

11. Экранный контракт

Каждый экран должен иметь 4 части:

1. render-функция экрана
2. keyboard-функция экрана
3. callback handlers для кнопок
4. state transition

Пример структуры:

```
async def show_some_screen(callback, state):
    data = await state.get_data()
    text = build_some_screen_text(data)
    keyboard = get_some_screen_keyboard(data)

    await callback.message.edit_text(
        text,
        reply_markup=keyboard,
        parse_mode="HTML",
    )

    await state.set_state(SomeStates.some_state)

def get_some_screen_keyboard(data):
    builder = InlineKeyboardBuilder()
    builder.button(text="✅ Продолжить", callback_data="some_continue")
    builder.button(text="⬅️ Назад", callback_data="some_back")
    builder.adjust(1)
    return builder.as_markup()
```

```
@router.callback_query(F.data == "some_continue")
async def some_continue(callback, state):
    await state.update_data(some_step_completed=True)
    await show_next_screen(callback, state)
    await callback.answer()
```

12. Что LLM-агент обязан сдать

LLM-агент должен создать не просто backend, а полный пользовательский продукт:

```
bot/states.py
bot/keyboards.py
bot/handlers/common.py
bot/handlers/generation.py
bot/handlers/payments.py
bot/handlers/admin.py
bot/handlers/image_analyzer.py
bot/miniapp.py
bot/main.py
bot/config.py
bot/database.py
bot/services/*
data/price.json
data/presets.json
static/uploads/
tests/
```

Минимальные тесты:

1. FSM states импортируются.
2. Главное меню собирается без ошибок.
3. Фото-флоу стартует через create_image_text_new.
4. Видео-флоу стартует через create_video_new.
5. Выбор модели обновляет FSM data.
6. Загрузка референса добавляет URL в FSM data.
7. Prompt запускает generation_task.
8. Недостаток баланса блокирует запуск.
9. Успешный webhook завершает task.
10. Повтор генерации восстанавливает request_data.
11. Payment webhook начисляет кредиты один раз.
12. Admin router доступен только admin_ids.

13. Критерий готовности UX

Проект считается готовым только если пользователь может пройти такие сценарии без ручных команд:

```
/start
→ Главное меню
→ Создать фото
→ выбрать модель
→ добавить или пропустить референсы
→ выбрать формат/качество/количество
→ написать prompt
→ получить результат
→ повторить или опубликовать

/start
→ Главное меню
→ Создать видео
→ выбрать модель
→ выбрать тип: text/imgtxt/video/avatar/motion
→ загрузить нужные файлы
→ выбрать формат/длительность/качество
→ написать prompt
→ получить результат

/start
→ Баланс
→ Пополнить
→ выбрать пакет
→ оплатить
→ получить бананы

/start
→ Лента
→ открыть чужую работу
→ повторить/ремикснуть

/admin
→ открыть админку
→ посмотреть статистику
→ изменить цены
→ создать промокод
→ сделать рассылку
```

Главное правило: новый проект должен копировать не старый бренд и не старый репозиторий, а проверенный UX/FSM-паттерн: экран → кнопки → state update → следующий экран → task → webhook → результат.